Tree and Binary Search Tree

Sungdeok (Steve) Cha

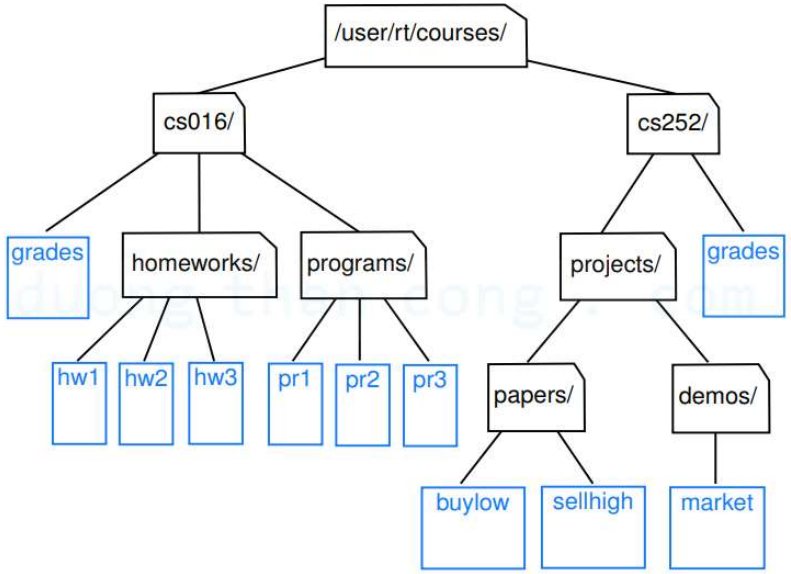
VinUni, CECS

Learning Objectives

- **Tree structure is very frequently used**
  - May provide O(log n) performance in search, insertion, and deletion
- **Understand binary search tree algorithms**
  - Other types of trees (e.g., B-trees) are sometimes used. Especially in DBMS implementations
- **Understand how tree, abstract data structure, is implemented**
  - Doubly linked list is usually used. Understand why.
  - Array may be used in certain types of trees. It is called “Heap”
- **Understand the need for height-balanced tree**
  - AVL tree
  - Understand why Red-Black tree is used in practice

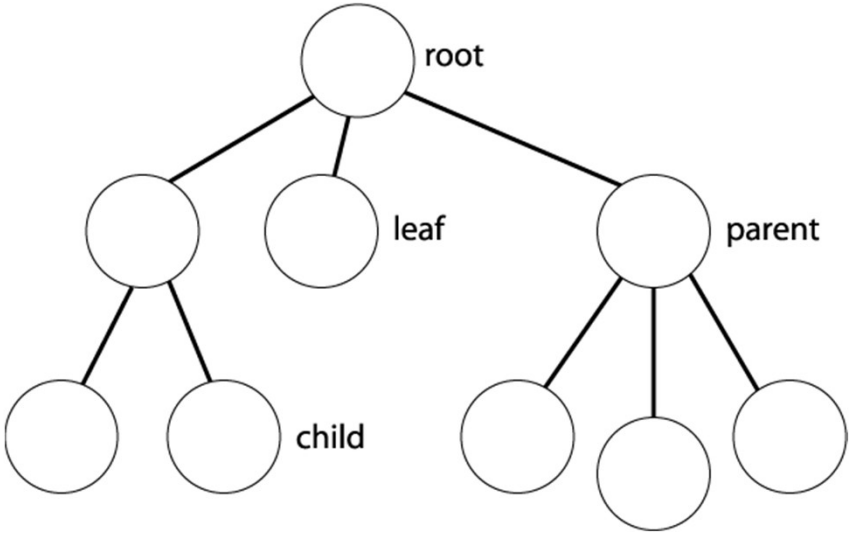
Tree

- **Non-linear data structure organized based on hierarchical relationship**
  - **A node has one predecessor (i.e., parent), but can h ave multiple successors (i.e., child)**
    - **For binary tree, up to two**
  - **The root of the tree is at the top!**



Tree Terminology

- **Root, Parent, Child**
- **Siblings: nodes with same parent**
- **Leaf vs internal nodes**
  - **Leaf node has no children**
- **Internal nodes: non-leaf nodes**



Level, Depth, Height, ...

In tree data structures, the terms **level, depth, and height** have specific meanings:

1. **Level**:
   - The level of a node represents its distance from the root node.
   - The root node is at **level 0**, its children are at **level 1**, their children are at **level 2**, and so on.
2. **Depth**:
   - The depth of a node is the number of edges (or steps) from the root node to that node.
   - It is equivalent to the **level** of that node.
   - Formula:
     $$\text{Depth}(N) = \text{Level}(N)$$
3. **Height**:
   - The height of a node is the number of edges (or steps) on the longest path from that node to a leaf node.
   - The height of a **leaf node** is **0** (since there are no edges below it).
   - The height of the **tree** is the height of the root node.

Level, Depth, Height, ...

```
      A (Level 0)
     / \
    B   C (Level 1)
   / \   \
  D   E   F (Level 2)
```

- **Depth of C** = 1 (one step from A)
- **Height of B** = 1 (longest path: B → D or B → E)
- **Height of A (Tree Height)** = 2 (longest path: A → B → D or A → C → F)

DIY

Summary:

Node	Level (Depth)	Height
A	0	3
B	1	2
C	1	1
D	1	2
E	2	0
F	2	1
G	2	0
H	2	1
I	2	0
J	3	0
K	3	0
L	3	0

Don’t Get Confused !!!

ChatGPT

“Appearance”
(Not about data contents)

Essential Topic (On how to store and organize data)

(Not now…)

📌 Summary Table

Binary Tree Type	Definition
Full Binary Tree	Each node has 0 or 2 children.
Complete Binary Tree	All levels are full except the last, filled left to right.
Perfect Binary Tree	Every internal node has 2 children, all leaves are at the same level.
Balanced Binary Tree	Left & right subtrees differ in height by at most 1.
Degenerate Tree	Every node has only one child (left or right).
Binary Search Tree (BST)	Left subtree < Root < Right subtree (sorted).
AVL Tree	Self-balancing BST with height difference ≤ 1.
Red-Black Tree	Self-balancing BST with red-black rules.
Threaded Binary Tree	Uses extra links to improve traversal.
B-Trees	Multi-child search tree used in databases.

🚀 Final Thoughts

- **For fast searching?** Use **BST, AVL Tree, or Red-Black Tree.**
- **For database indexing?** Use **B-Trees.**
- **For priority queues?** Use **Complete Binary Tree (Heap).**
- **For memory efficiency?** Use **Threaded Binary Tree.**

Would you like a **Java implementation of any of these trees?** 😊 🚀

Full Binary Tree

- **Has nothing to do with "contents"**
- **We will learn later that it is NOT a binary search tree**
  - OK if it does not make sense now

1 Full Binary Tree

📌 Definition:

- Every node has **0 or 2 children** (no node has only one child).
- A leaf node has **no children**, while an internal node has **exactly two children**.

✅ Example:

markdown

```
    1
   / \
  2   3
 / \
4   5
```

🚀 Properties:

- If a tree has `n` leaf nodes, it has `n - 1` internal nodes.

Complete Binary Tree

- **Useful as it can be easily stored in array**
  - Location of child nodes can be computed
  - Root at [0]
- **That's why some tree-based data structures (e.g., MaxHeap, MinHeap) enforce complete binary tree requirements**

2 Complete Binary Tree

📌 **Definition:**

- All levels **except the last** are **completely filled.**
- The last level **must be filled from left to right.**

✅ **Example:**

markdown

```
      1
    /   \
   2     3
  / \   /
 4   5 6
```

🚀 **Properties:**

- Used in **Binary Heaps.**
- Efficient storage in **arrays** (heap representation).

explain array representation of complete binary tree

📌 Array Representation of a Complete Binary Tree

A **Complete Binary Tree** is a **binary tree** where **all levels** are completely filled except possibly the **last level**, which is filled **from left to right**.

Instead of using **pointers** (linked structure), we can store a **complete binary tree** in an **array** using **index-based relationships**.

🔢 Indexing Rules for a Complete Binary Tree in an Array

For a **node at index** `i` in a **0-based array**, the relationships are:

1. **Parent of node at index** `i` = `(i - 1) / 2`
2. **Left Child of node at index** `i` = `2i + 1`
3. **Right Child of node at index** `i` = `2i + 2`

For a **1-based index array**, the formulas change:

1. **Parent of node at index** `i` = `i / 2`
2. **Left Child of node at index** `i` = `2i`
3. **Right Child of node at index** `i` = `2i + 1`

➡️ Array Representation (0-based index)

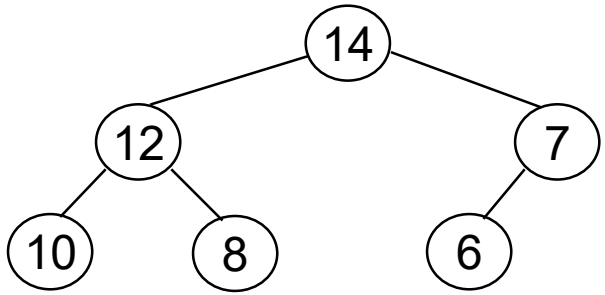
```makefile
Index:  0   1   2   3   4   5
Value:  [1,  2,  3,  4,  5,  6]
```

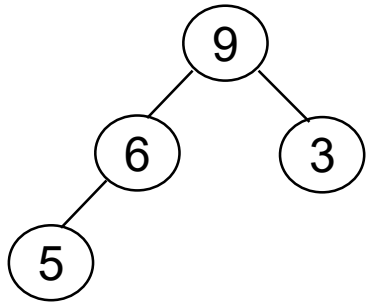
- Root (`1`) is at index `0`
- Left Child of `1` (2) → index `(2*0 + 1) = 1`
- Right Child of `1` (3) → index `(2*0 + 2) = 2`
- Left Child of `2` (4) → index `(2*1 + 1) = 3`
- Right Child of `2` (5) → index `(2*1 + 2) = 4`
- Left Child of `3` (6) → index `(2*2 + 1) = 5`

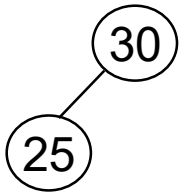
Heap

- **Do you see why array is used to represent a heap structure?**

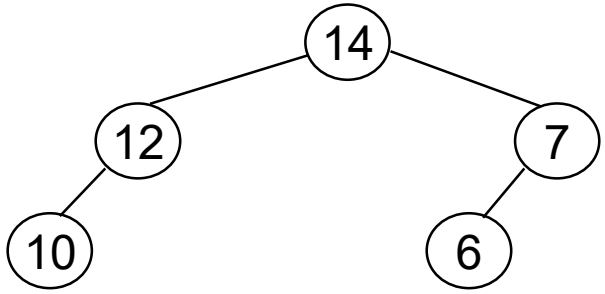
MaxHeap

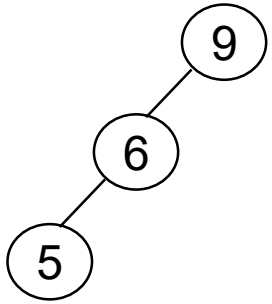


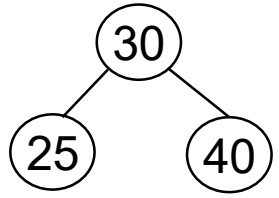




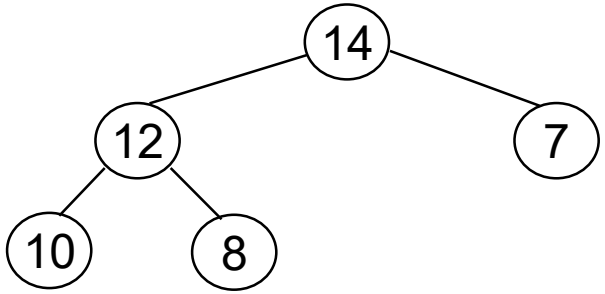
Not a MaxHeap! Why?



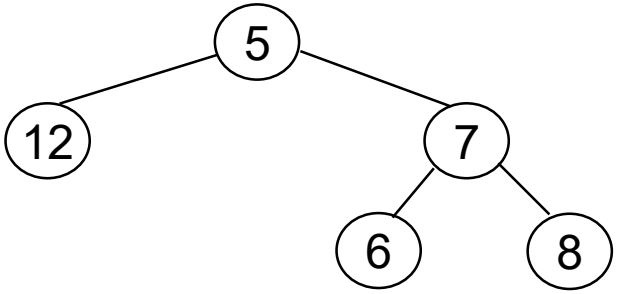




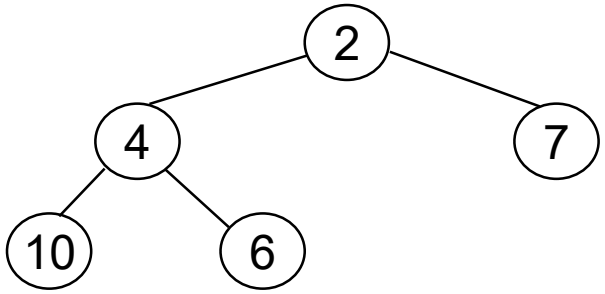
Full/Complete Tree and Heap



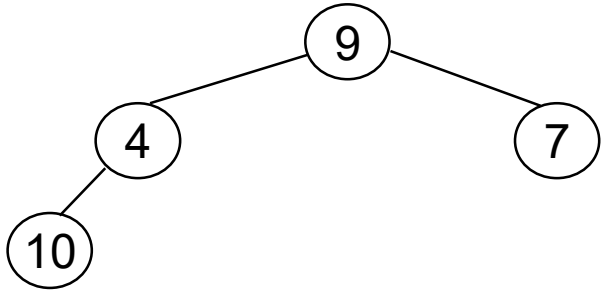
Full Binary Tree? Y
Complete Binary Tree? Y
MaxHeap? Y
MinHeap? N



Full Binary Tree? Y
Complete Binary Tree? N
MaxHeap? N
MinHeap? N



Full Binary Tree? Y
Complete Binary Tree? Y
MaxHeap? N
MinHeap? Y



Full Binary Tree? N
Complete Binary Tree? Y
MaxHeap? N
MinHeap? N

0-based? 1-based?

- **Nothing terribly complex or important**

1 0-Based Indexing (Most Common)

- The **first element** is at **index** `0`.
- The **second element** is at **index** `1`, and so on.

✅ **Example (0-based index array):**

makefile

```
Array:   [10, 20, 30, 40, 50]
Index:     0   1   2   3   4
```

👉 In Java, C, Python, C++, JavaScript, and most modern languages, **arrays are 0-based**.

📌 **Formula to access elements in a 0-based array:**

- `arr[i]` gives the **(i+1)th element** of the array.
- **First element** → `arr[0]`
- **Last element** (for `n` elements) → `arr[n-1]`

---

2 1-Based Indexing (Less Common)

- The **first element** is at **index** `1`.
- The **second element** is at **index** `2`, and so on.

📌 Advantages of Using Arrays for Complete Binary Trees

- ✅ **Memory Efficient:** No extra space needed for pointers.
- ✅ **Fast Access:** Parent, left, and right child can be computed in **O(1)** time.
- ✅ **Used in Heaps: Binary Heaps** (Min Heap & Max Heap) use this representation.

Stay alert!!!

📌 Limitations of Array-Based Representation

- ❌ **Wastes Space for Sparse Trees:** If the tree is not complete, many array slots are unused.
- ❌ **Insertion & Deletion Complexity:** Operations may require shifting elements (O(n) time).
- ❌ **Not Suitable for Arbitrary Trees:** If the tree is not complete, this method is inefficient.

🚀 Summary

- **Complete Binary Trees** are efficiently stored in arrays using **index-based calculations**.
- **Heap Data Structures** use this technique.
- **Operations are fast** (O(1) for parent/child retrieval) but **insertion/deletion may be costly**.

Perfect Binary Tree

3 Perfect Binary Tree

📌 **Definition:**

- **All internal nodes** have **exactly 2 children**.
- **All leaf nodes** are at the **same level**.

✅ **Example:**

markdown

```
      1
    /   \
   2     3
  / \   / \
 4   5 6   7
```

🚀 **Properties:**

- Number of **nodes** = `2^h - 1` (where `h` is height).
- Number of **leaf nodes** = `2^(h-1)`.
- Used in **binary search optimizations**.

Hmm...

- **Is full binary tree always a complete binary tree?**
- **Is complete binary tree always a full binary tree?**
- **Is perfect binary tree a full binary tree?**
- **...**

1 Full Binary Tree

📌 **Definition:**

- Every node has **0 or 2 children** (no node has only one child).
- A leaf node has **no children**, while an internal node has **exactly two children**.

✅ **Example:**

```
markdown
    1
   / \
  2   3
 / \
4   5
```

🚀 **Properties:**

- If a tree has `n` leaf nodes, it has `n - 1` internal nodes.

2 Complete Binary Tree

📌 **Definition:**

- All levels **except the last** are **completely filled**.
- The last level **must be filled from left to right**.

✅ **Example:**

```
markdown
    1
   / \
  2   3
 / \ /
4  5 6
```

🚀 **Properties:**

- Used in **Binary Heaps**.
- Efficient storage in **arrays** (heap representation).

3 Perfect Binary Tree

📌 **Definition:**

- **All internal nodes** have **exactly 2 children**.
- **All leaf nodes** are at the **same level**.

✅ **Example:**

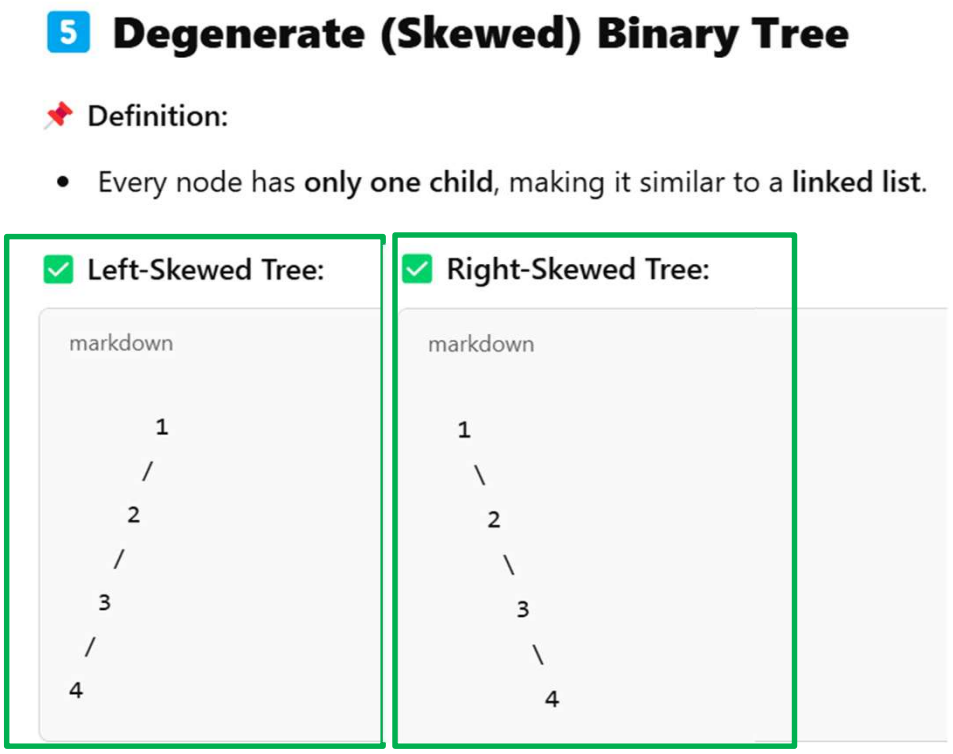
```
markdown
      1
    /   \
   2     3
  / \   / \
 4   5 6   7
```

🚀 **Properties:**

- Number of **nodes** = `2^h - 1` (where `h` is height).
- Number of **leaf nodes** = `2^(h-1)`.
- Used in **binary search optimizations**.

Skewed Binary Tree

- **Becomes essentially equivalent to linked list**
- **Applications do not have control over input orders**
  - Need smart algorithms to prevent tree from being skewed
  - AVL tree or Red-Black tree



Binary Search Tree (BST)

- **Intuitive Algorithm**
  - Could result in skewed (e.g., inefficient) BST

✅ Right-Skewed Tree:

markdown

```
1
 \
  2
   \
    3
     \
      4
```

6 Binary Search Tree (BST)

📌 **Definition:**

- A binary tree where:
  - **Left subtree** contains values **less than** root.
  - **Right subtree** contains values **greater than** root.
  - No **duplicate** values.

✅ **Example:**

markdown

```
      50
     /  \
   30    70
  /  \   / \
 20   40 60 80
```

🚀 **Properties:**

- Efficient **search, insert, delete** in **O(log n)** (if balanced).
- Used in **databases, indexing, and searching algorithms.**

which data structeure is most frequently used to implement binary search tree? why?

The **most frequently used data structure to implement a Binary Search Tree (BST)** is a **linked structure** (also known as a **node-based structure**). Here's why:

✅ Why a Linked Structure is Preferred:

1. **Dynamic Size:**
   - Nodes can be added or removed easily without reallocating or resizing memory (unlike arrays).
   - BSTs grow and shrink as needed, and a linked structure handles this efficiently.
2. **Efficient Insertion and Deletion:**
   - Operations like insertion, deletion, and searching are implemented via pointer manipulation, which is efficient in a node-based setup.
3. **Flexibility in Tree Structure:**
   - BSTs often become unbalanced, and restructuring (e.g., rotations) is easier with pointers rather than shifting array elements.
4. **Memory Usage:**
   - Linked structures use memory as needed (for each node), whereas arrays may waste memory if not fully utilized.

💡 Alternative (Less Common): Array-based Representation

Used in **heap** structures (like binary heaps), not typically for BSTs. It's less efficient for BSTs due to:

- Wasted space (especially in unbalanced trees)
- Difficulties in dynamic insertion and deletion

LinkedList and Tree Structure

Advantages

- Dynamic Size Allocation
- Efficient Insertions and Deletions
- Flexible Structure
- Memory Utilization
- Easy Implementation of Parent and Child Links

Disadvantages

- Extra Memory Overhead
- Slower Access Time
- Complex Implementation
- Cache Inefficiency
- Garbage Collection Issues (in some languages)
  - Not in Java
  - Manual deallocation required in C or C++

BST and Java Code

Java has great support for implementing Binary Search Trees (BST), though it doesn't come with a built-in `BinarySearchTree` class in the standard library. You'll typically implement a BST manually or use third-party libraries like Apache Commons or Google's Guava if you want tree-like data structures with specific behaviors.

```java
// Class representing a node in the BST
class TreeNode {
    int key;
    TreeNode left, right;

    public TreeNode(int item) {
        key = item;
        left = right = null;
    }
}

// BST class with basic operations
public class BinarySearchTree {
    TreeNode root;

    // Constructor
    BinarySearchTree() {
        root = null;
    }
```

```java
    // Public insert method
    void insert(int key) {
        root = insertRec(root, key);
    }

    // Recursive insert method
    TreeNode insertRec(TreeNode root, int key) {
        if (root == null) {
            root = new TreeNode(key);
            return root;
        }

        if (key < root.key)
            root.left = insertRec(root.left, key);
        else if (key > root.key)
            root.right = insertRec(root.right, key);

        return root;
    }
```

BST and Java Code

```
41        // Public search method
42        boolean search(int key) {
43            return searchRec(root, key);
44        }
45
46        // Recursive search method
47        boolean searchRec(TreeNode root, int key) {
48            if (root == null)
49                return false;
50            if (root.key == key)
51                return true;
52            return key < root.key
53                ? searchRec(root.left, key)
54                : searchRec(root.right, key);
55        }
56
57        // In-order traversal (prints the BST in sorted order)
58        void inorder() {
59            inorderRec(root);
60            System.out.println();
61        }
62
63        void inorderRec(TreeNode root) {
64            if (root != null) {
65                inorderRec(root.left);
66                System.out.print(root.key + " ");
67                inorderRec(root.right);
68            }
69        }
```

```
71        // Main method for testing
72        public static void main(String[] args) {
73            BinarySearchTree bst = new BinarySearchTree();
74
75            // Insert nodes
76            bst.insert(50);
77            bst.insert(30);
78            bst.insert(70);
79            bst.insert(20);
80            bst.insert(40);
81            bst.insert(60);
82            bst.insert(80);
83
84            // Print in-order traversal
85            System.out.print("In-order traversal: ");
86            bst.inorder();
87
88            // Search for a key
89            System.out.println("Search 60: " + bst.search(60)); // true
90            System.out.println("Search 90: " + bst.search(90)); // false
91        }
92    }
93
```

Java and Binary Tree

does java have built-in binary tree class?

Java **does not** have a built-in **Binary Tree** class specifically, but it provides the `TreeSet` and `TreeMap` classes in the **Java Collections Framework**, which internally use a **self-balancing Red-Black Tree** (a type of Binary Search Tree).

We will study "simpler" AVL tree instead

Alternatives for Binary Trees in Java

1. **TreeSet (For Unique Sorted Elements)**
   - Implements a **self-balancing Binary Search Tree (Red-Black Tree)**.
   - Automatically maintains elements in sorted order.
   - **Does not allow duplicates.**

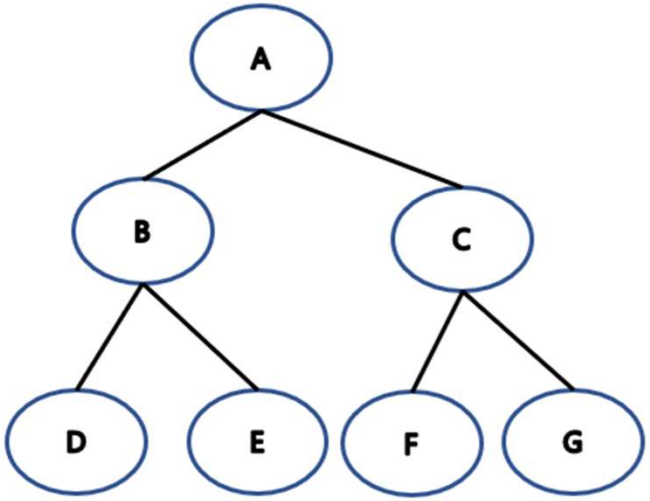
java

```java
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet<Integer> treeSet = new TreeSet<>();
        treeSet.add(50);
        treeSet.add(30);
        treeSet.add(70);
        treeSet.add(20);
        treeSet.add(40);
```

Binary Tree Traversal

- **Preorder**
- **Inorder**
- **Postorder**



1. Depth-First Traversal (DFS)

These methods explore as deep as possible along each branch before backtracking.

- **Preorder Traversal (Root → Left → Right)**
  - Visit the root node.
  - Recursively traverse the left subtree.
  - Recursively traverse the right subtree.
  - **Example:** `A → B → D → E → C → F → G`
- **Inorder Traversal (Left → Root → Right)**
  - Recursively traverse the left subtree.
  - Visit the root node.
  - Recursively traverse the right subtree.
  - **Example:** `D → B → E → A → F → C → G`
  - **Use Case:** This traversal results in a sorted sequence for a Binary Search Tree (BST).
- **Postorder Traversal (Left → Right → Root)**
  - Recursively traverse the left subtree.
  - Recursively traverse the right subtree.
  - Visit the root node.
  - **Example:** `D → E → B → F → G → C → A`

One of KU's Data Structure Exam Question

- **Credit to Prof. Won-Ki Jeong**
- **Draw a binary tree whose inorder and postorder traversal sequences are as follows:**
  - Inorder : "A D E F C I H B G"
  - Postorder : "A D F I H C G B E"

Binary Search Tree (BST)

- Left child contains smaller values than the parent
- Right child contains larger values than the parent
- Duplication is NOT allowed

- **Intuitive algorithm ;)**

insertion algorithm into binary tree

Inserting a node into a **Binary Tree** depends on whether the tree is a **Binary Search Tree (BST)** or a general **Binary Tree**.

Insertion in a Binary Search Tree (BST)

A **BST** maintains the property:

- Left child contains smaller values than the parent.
- Right child contains larger values than the parent.

Algorithm for Inserting into a BST

1. Start at the root.
2. If the new value is smaller than the current node, go to the left subtree; if larger, go to the right.
3. If an empty spot is found, insert the new node there.

deletion algorithm on binary search tree

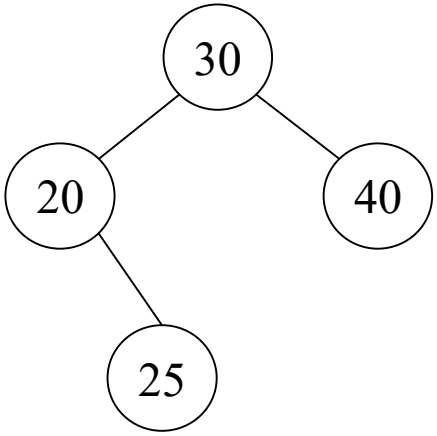
Deletion Algorithm in a Binary Search Tree (BST)

Deleting a node in a **Binary Search Tree (BST)** involves three main cases:

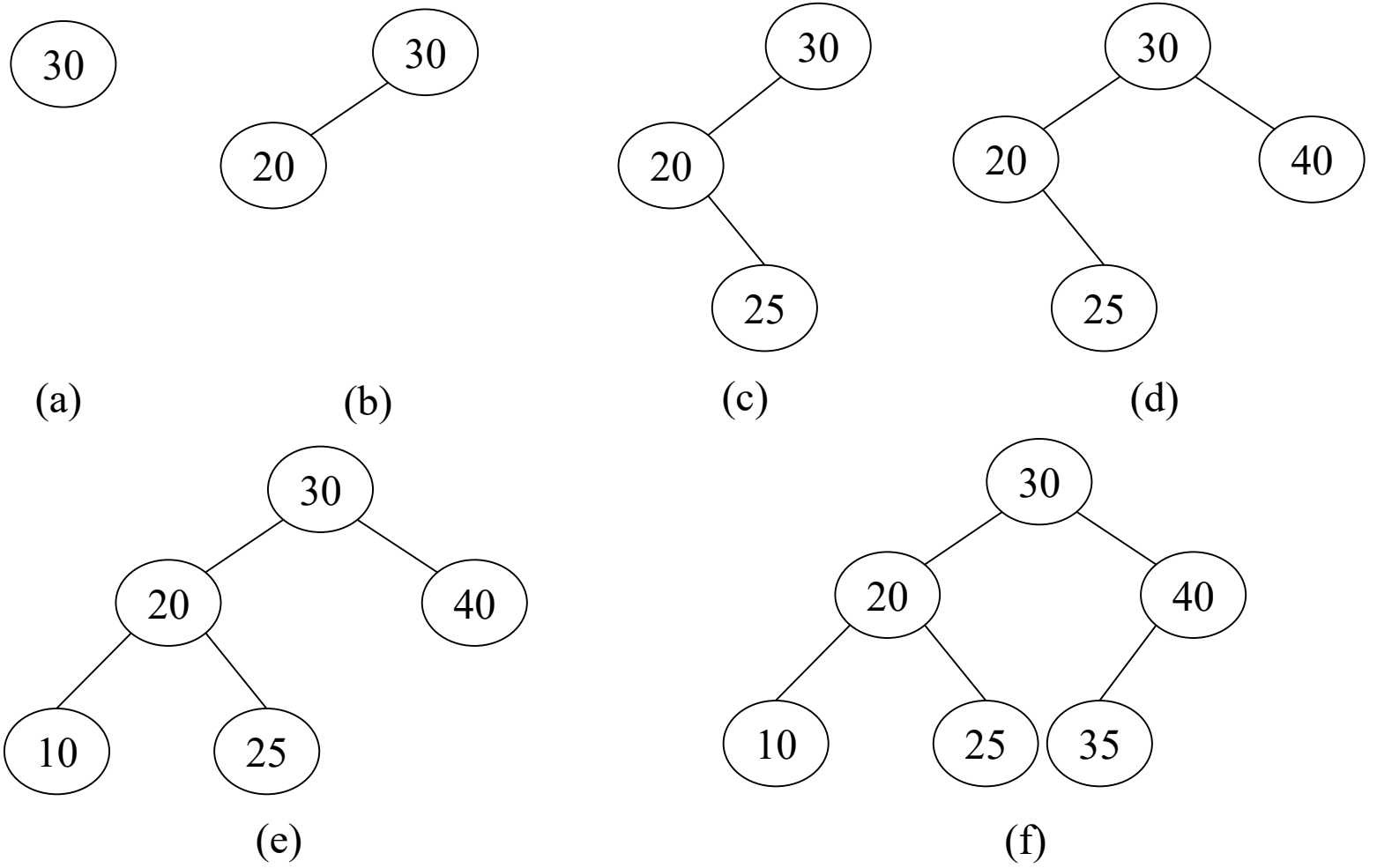
1. **Node has no children (leaf node)** → Simply remove it.
2. **Node has one child** → Replace the node with its child.
3. **Node has two children** → Find the **inorder successor** (smallest node in the right subtree) or **inorder predecessor** (largest node in the left subtree) and replace the node with it.

Insertion in Binary Search Trees

```
insert(t, x):
◀ t : root node, x : key to insert
    if (t=NIL)
        r.key ← x                ◀ r: new node
        return r
    if (x < t.key)
        t.left ← insert(t.left, x)
        return t
    else t.right ← insert(t.right, x)
        return t
```



Examples of Insertion



Deletion in Binary Search Trees

There are three cases

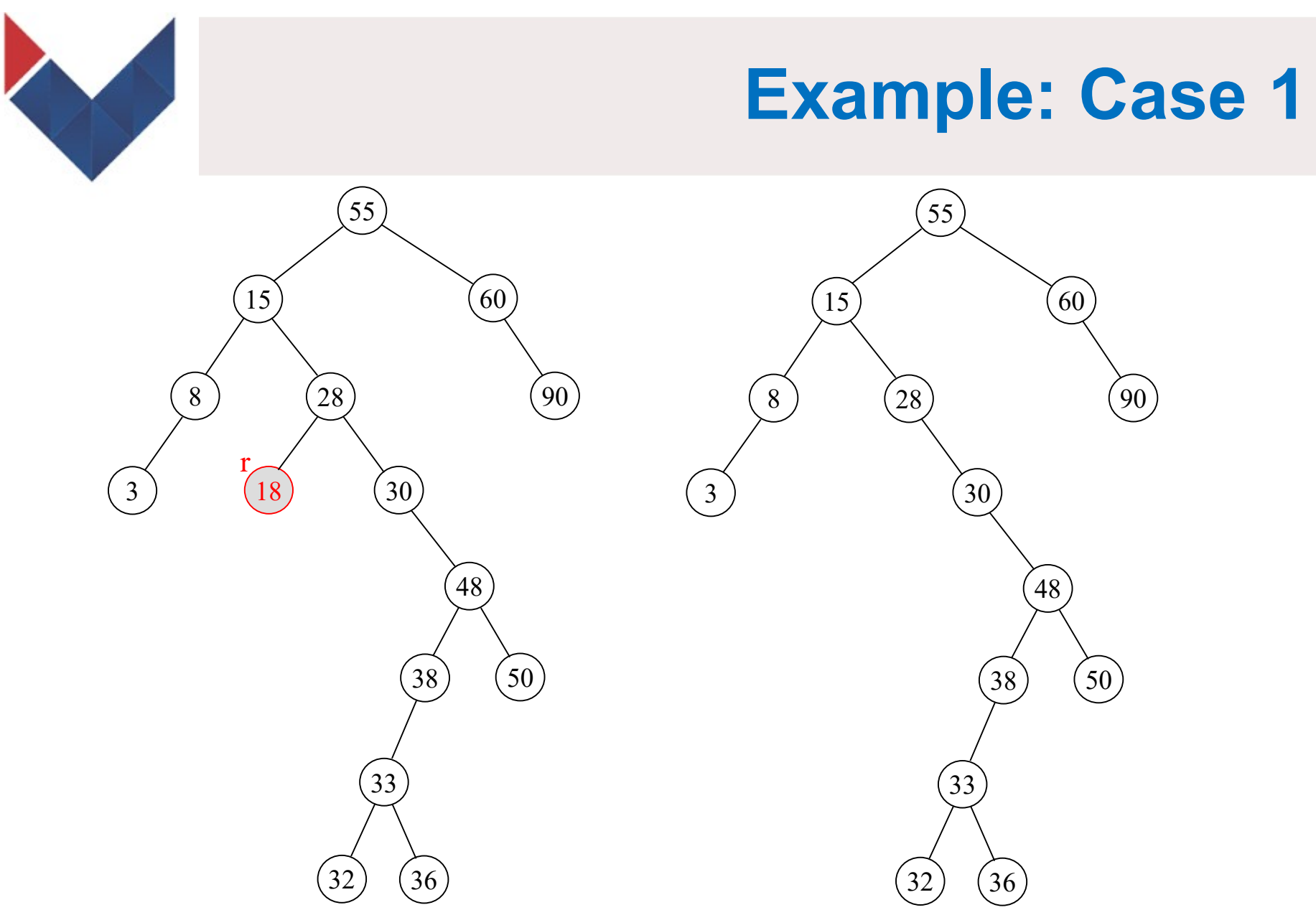
- Case 1 : r is a leaf node
- Case 2 : r has only one child
- Case 3 : r has two children

t: root node

r: node to deleted

Deletion in Binary Search Trees

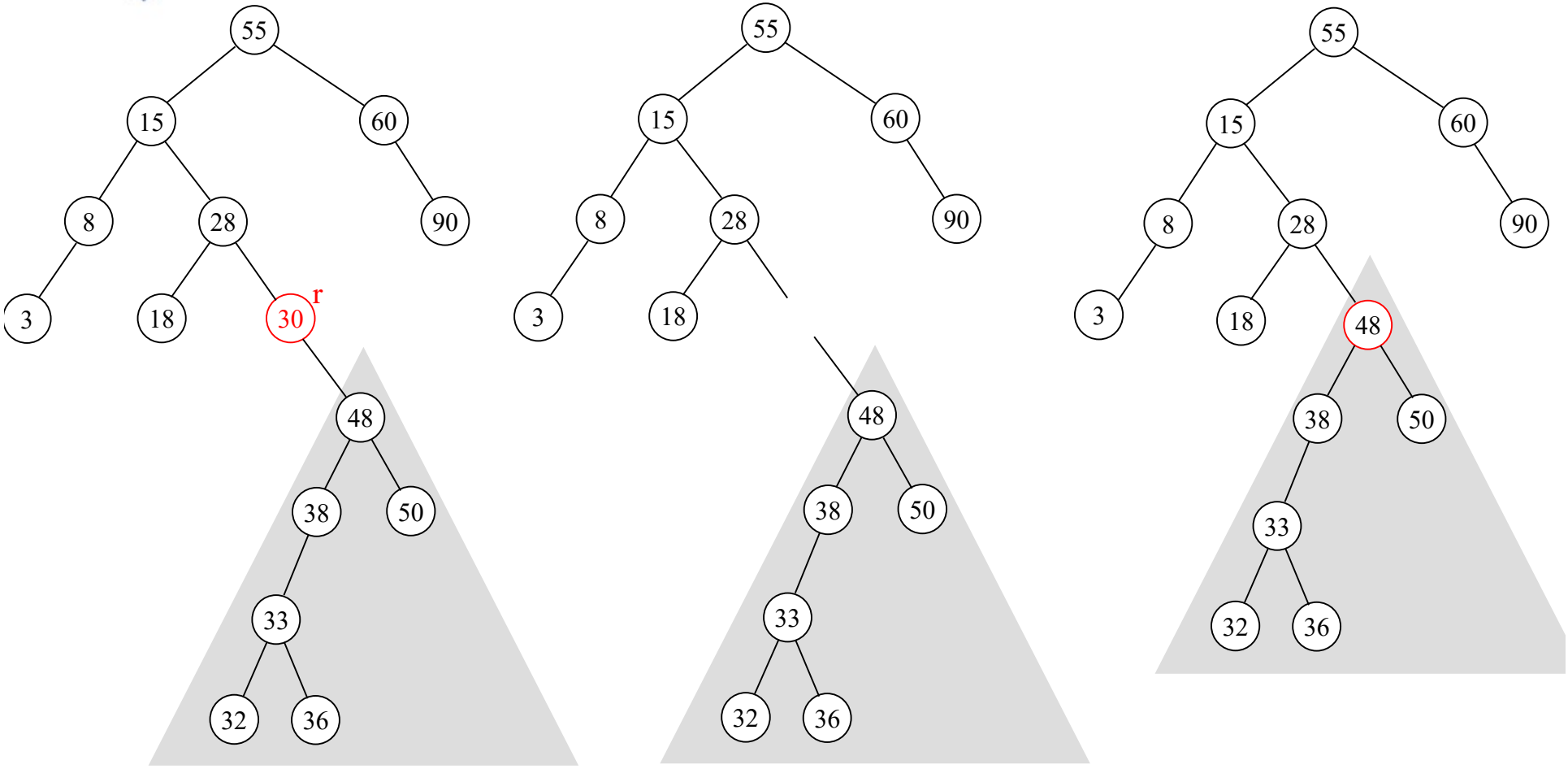
```
sketch_delete(t, r):
◀ t : root node, r : node to delete
    if (r is a leaf node)                          ◀ Case 1
        Just throw away r
    else if (r has only one child)                 ◀ Case 2
        Let r's parent links to the (only) child of r
    else                                           ◀ Case 3
        Remove the minimum node s of r's right subtree,
        and copy the key of s to node r
```



(a) r has no child

(b) Simply throw away r

Example: Case 1



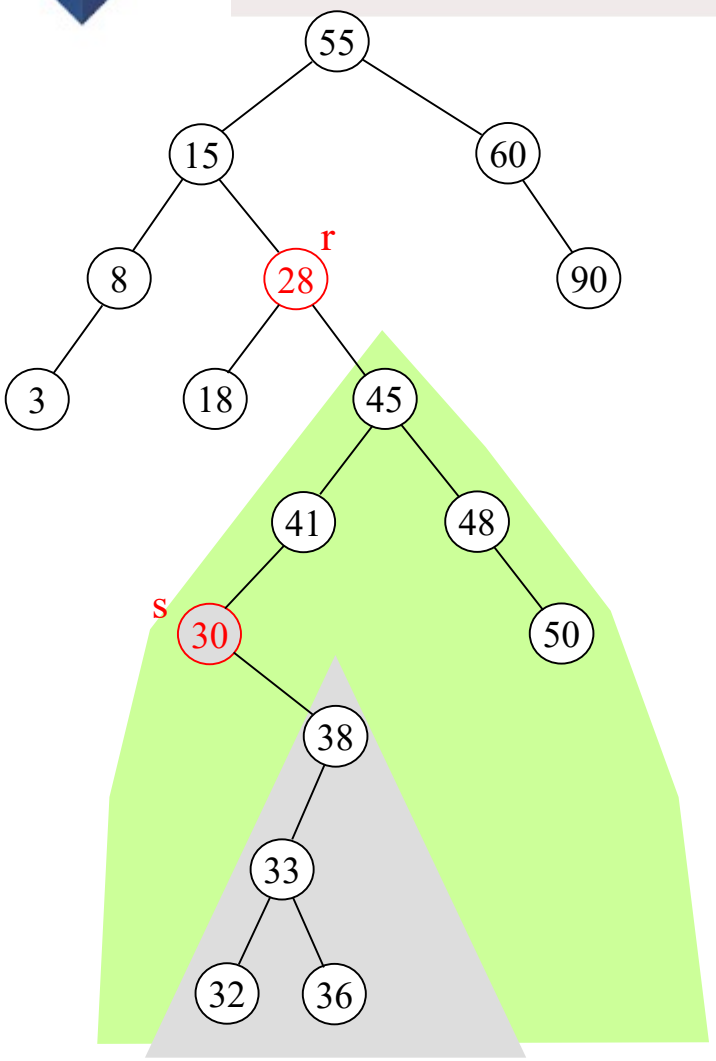
Example: Case 2

(a) r has only one child

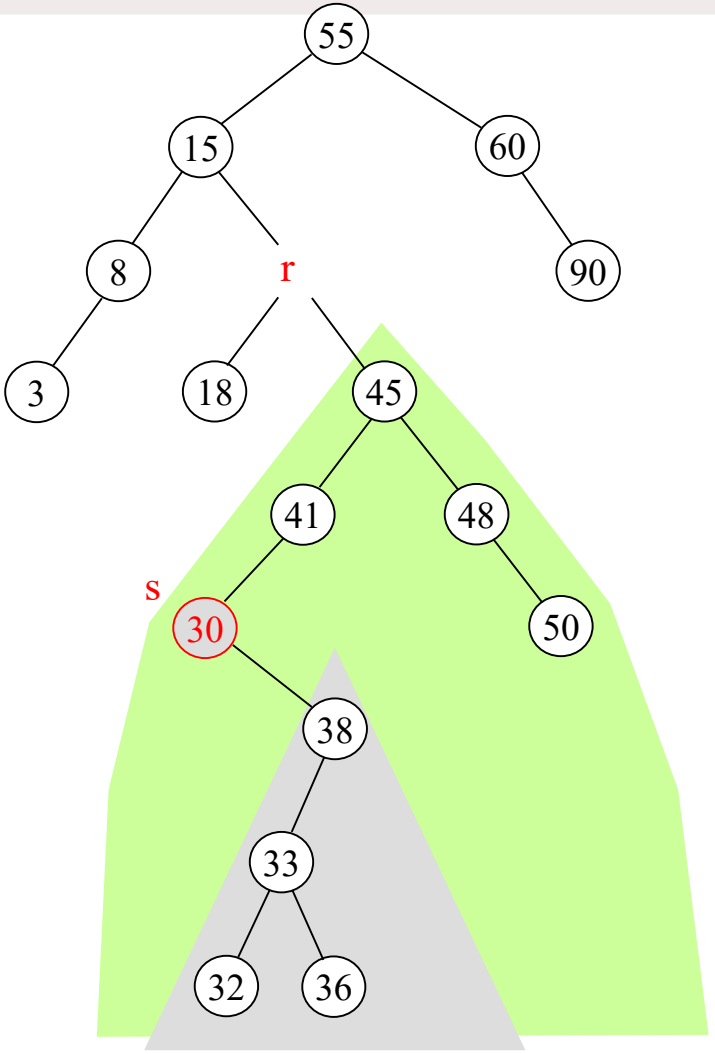
(b) Remove r

(c) Put r's (only) child at r's location

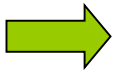
Example: Case 3

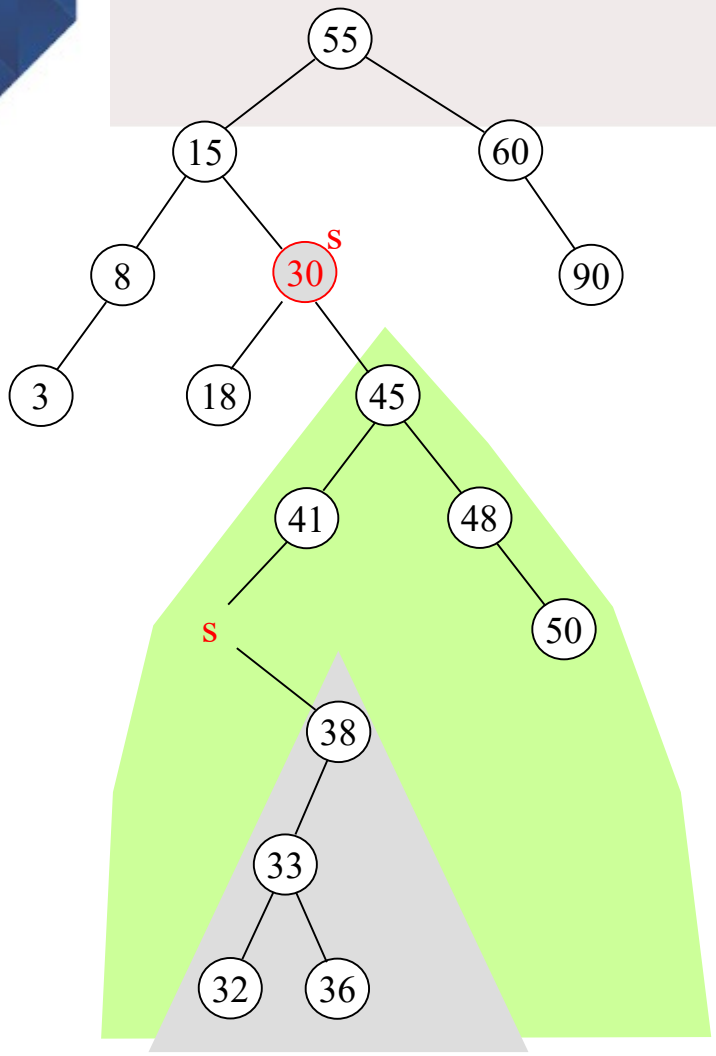


(a) Find r's inorder successor s

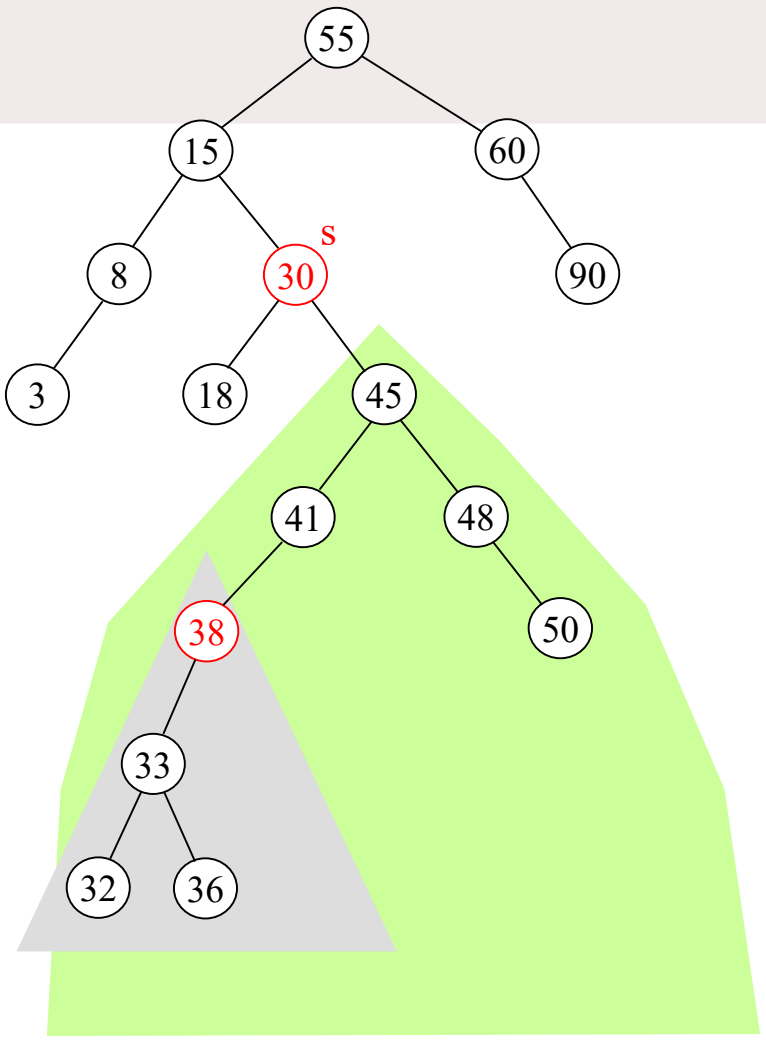


(b) Remove r (imaginary removal)





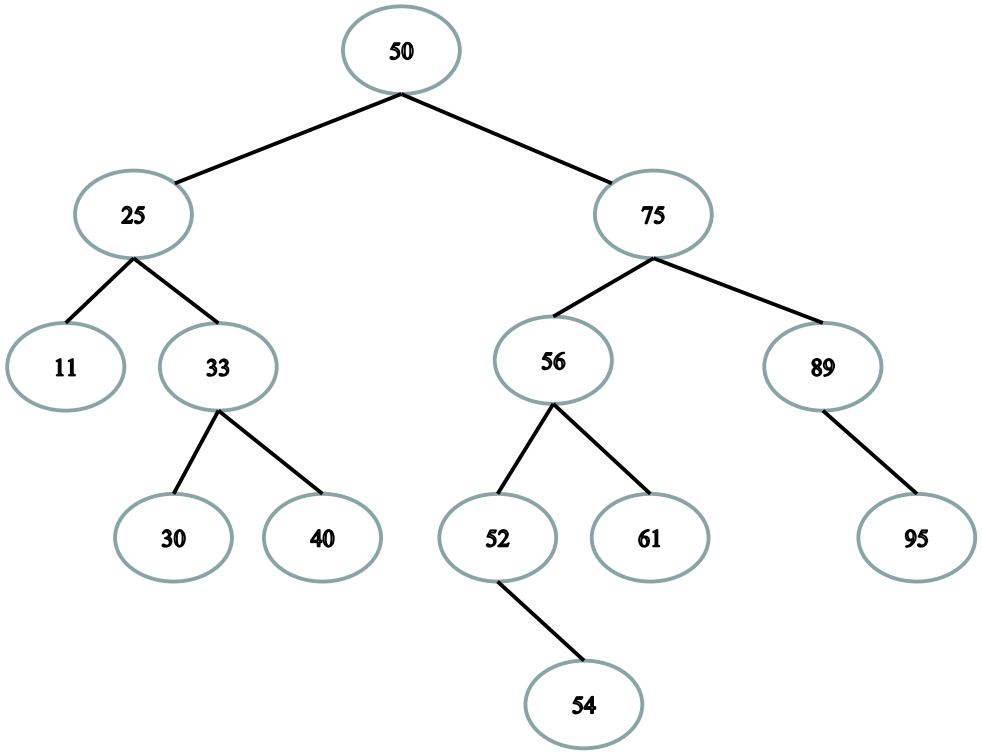
(c) Move s to r's location
(copy s to r)



(d) Move s's (only) child to s's location

DIY

- **Insert 2 ?**
- **Insert 55 ?**
  - Tree might become “less balanced”, but it is still a BST
- **Delete 30 ?**
- **Delete 52 ?**
- **Delete 50 ?**
  - Choose inorder predecessor?
  - Choose inorder successor?



is duplication generally allowed in binary search tree?

Ah, the classic "duplicates in a BST" question — love it. 😄
So here's the lowdown:

🎄 Is Duplication Generally Allowed in a Binary Search Tree (BST)?

🔧 Short Answer:

> 🔸 **Yes, duplicates *can* be allowed in a BST**, but it depends on the implementation and use case.

Question	Answer
Are duplicates allowed in a BST?	✅ Yes, **if you define a strategy**
Is there one standard way?	❌ No — depends on your needs
What's the safest?	➕ Store counts or allow on one side

1. Allow duplicates on one side (usually right)

```
Left: strictly less than
Right: greater than or equal to
```

This tweaks the BST rule to:

```
Left < N
Right ≥ N
```

- ✅ Simple and preserves the BST structure.
- 🔁 Still works for most BST operations (insert, search, delete).

2. Count duplicates at each node

Each node stores a `count` of how many times that value appears.

- ✅ Useful when you're more interested in frequency than structure.
- 🧱 Slightly more memory per node, but no change to tree shape.

3. Store duplicates in a list or multiset at each node

- ✅ Great if you want to keep track of actual objects or metadata with same keys.
- 📦 More complex structure per node.

suggest some problems where binary search tree is a useful data structure

Absolutely! A **Binary Search Tree (BST)** is a powerful data structure when you want to maintain elements in **sorted order** while supporting **fast insert, delete, and search** operations — all ideally in **O(log n)** time.

Here's a list of **classic problems and use cases** where BSTs (or variations like AVL or Red-Black Trees) are the most useful:

🌳 Binary Search Tree Problem Ideas

1. **Validate Binary Search Tree**

- **Prompt:** Check if a given binary tree is a valid BST.
- **Why BST?** Recursively verify that each node's value lies in a valid range.

---

2. **Lowest Common Ancestor in a BST**

- **Prompt:** Find the lowest common ancestor of two nodes in a BST.
- **Why BST?** Use the property that all left values < node < all right values.

Another KU's Data Structure Exam Question

- Credit to Prof. Won-Ki Jeong
- Assume that you insert a new element x to a binary search tree that does not contain x. After inserting x, you delete x immediately from the tree. If you repeat this insertion-deletion operation several times (each time with different value x), would the tree be same as the initial one? If yes, then explain why. If no, then give a counter example.

Another KU's Data Structure Exam Question

- Credit to Prof. Won-Ki Jeong
- Assume that you delete a leaf node x from a binary search tree and insert x back to the tree immediately. If you repeat this deletion-insertion operation several times (each time with different x value), would the tree be same as the initial one? If yes, then explain why. If no, then give a counter example.

- Assume that you delete a non-leaf node x from a binary search tree and insert x back to the tree immediately. If you repeat this deletion-insertion operation several times (each time with different x value), would the tree be same as the initial one? If yes, then explain why. If no, then give a counter example.

Potential Questions

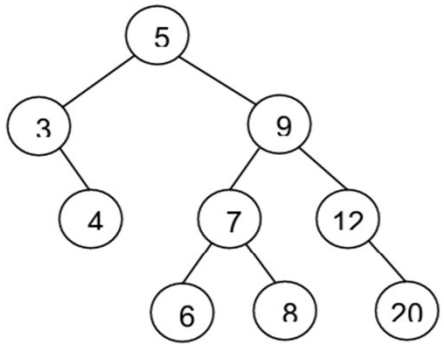
Part c. [3 points] Draw the binary tree given in this array.

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13
element	**9**	**3**	**10**	**1**	**6**		**14**			**4**	**7**			**13**

What do you have to do to convert this tree to AVL? Show the tree after appropriate rotations. Identify what type of rotation?

Now insert a new node (8) in this tree. Show the tree after appropriate rotations. Identify what type of rotation?

The binary search tree shown below was constructed by inserting a sequence of items into an empty tree.

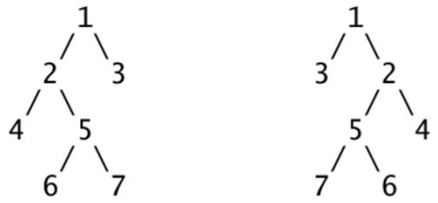


Which of the following input sequences will *not* produce this binary search tree?

A. 5 3 4 9 12 7 8 6 20
B. 5 9 3 7 6 8 4 12 20
C. 5 9 7 8 6 12 20 3 4
D. 5 9 7 3 8 12 6 4 20
E. 5 9 3 6 7 8 4 12 20

Potential Questions

b. *10 points*
Write a Java method named `mirror()` that takes a reference to the root node of a binary tree and creates a new tree (with its own nodes) that is the mirror image of the original tree. For example: if `root` is a reference to the root of the tree on the left below, then the return value of `mirror(root)` would be a reference to the root of the tree on the right below.
Hint: This method is much easier to write if you use recursion.




```
public static Node mirror(Node root) {
```

3. If a binary search tree is not allowed to have duplicates, there is more than one way to delete a node in the tree when that node has two children. One way involves choosing a replacement node from the left subtree. If this is done, which node are we looking for?

   A. the largest node in the subtree
   B. the smallest node in the subtree
   C. the root of the left subtree
   D. the next-to-smallest node in the subtree
   E. it doesn't matter – any node in the left subtree will do

E.1 [2] In the order given, insert the numbers 15, 65, 59, 68, 42, 40, 80, 50, 65 into an initially empty binary search tree.

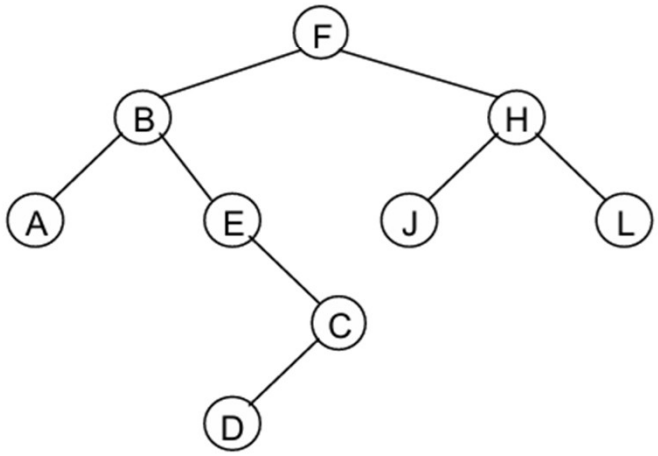
Potential Questions

Final Examination E&CE 250 W'01

PROBLEM 4 [16]

A. Trees

1.Consider the binary tree shown below. For each of the traversals listed, give the order in which the nodes are visited.



preorder									
inorder									
postorder									
breadth-first									


THANK YOU!